
aliases: - 02-slice

theory_02: 切片 — 从序列里取一段

读完约 4 分钟。

相关笔记: [列表深入操作](#) · [字符串方法](#)

语法

序列[开始位置 : 结束位置 : 步长]

start : stop : step

三条铁律: 1. 包含 start, 不包含 stop (取到 stop 的前一个位置) 2. 省略 start 默认从 0 开始 3. 省略 stop 默认到最后

字符串切片

```
s = "零一二三四五六七八九"
```

```
s[0:3]    # "零一二"    —— 索引0, 1, 2, 不含索引3
```

```
s[3:6]    # "三四五"    —— 索引3, 4, 5
```

```
s[:4]     # "零一二三"   —— 省略start, 从0开始
```

```
s[6:]     # "六七八九"   —— 省略stop, 到最后
```

```
s[:]      # 全部        —— 等于复制整个序列
```

列表切片

```
nums = [10, 20, 30, 40, 50, 60, 70]

nums[0:3]    # [10, 20, 30]
nums[:3]     # [10, 20, 30]    —— 省略start
nums[4:]     # [50, 60, 70]    —— 省略stop
```

为什么 stop 不包含

```
# stop - start = 取出的元素个数
s[0:3]    # 3 - 0 = 3 个元素: "零一二"
s[2:5]    # 5 - 2 = 3 个元素: "三四五"

# 让相邻切片无缝衔接
s[0:3]    # "零一二"
s[3:6]    # "三四五"    ← 上一段的stop = 下一段的start
```

步长（第三个参数）

```
s = "零一二三四五六七八九"

s[::2]    # "零二四六八"    —— 从头到尾，隔一个取一个
s[::3]    # "零三六九"      —— 隔两个取一个
s[::-1]   # "九八七六五四三二零" —— 步长-1 = 反转！
```

负索引

```
s = "零 一 二 三 四 五 六 七 八 九"
索引: 0  1  2  3  4  5  6  7  8  9
负索引:-10 -9 -8 -7 -6 -5 -4 -3 -2 -1
```

越界行为（重要）

```
s = "零一二三四五六七八九"

s[100]      # IndexError —— 索引越界，报错
s[0:100]    # 不报错，自动截断到末尾 → 输出完整字符串
```

规则：单元素索引越界 = 报错。切片越界 = 自动截断，不报错。

步长方向陷阱

默认步长是 `+1`（从左往右走），所以 **start 必须在 stop 的左边**才能取到值。

```
s[-3:-1]    # "七八" —— start在stop左边，正常
s[-1:-3]    # "" —— start在stop右边，步长为正→走不到，返回空串

# 想反向取值必须用负步长：
s[-1:-3:-1] # "九八" —— 步长-1，往回走
```

备考相关

- [\[\[EXAM_PREP/day02/00_今日任务\]\]](#) — Day 2 字符串与组合数据类型